처음 설치하는 분은 여기부터

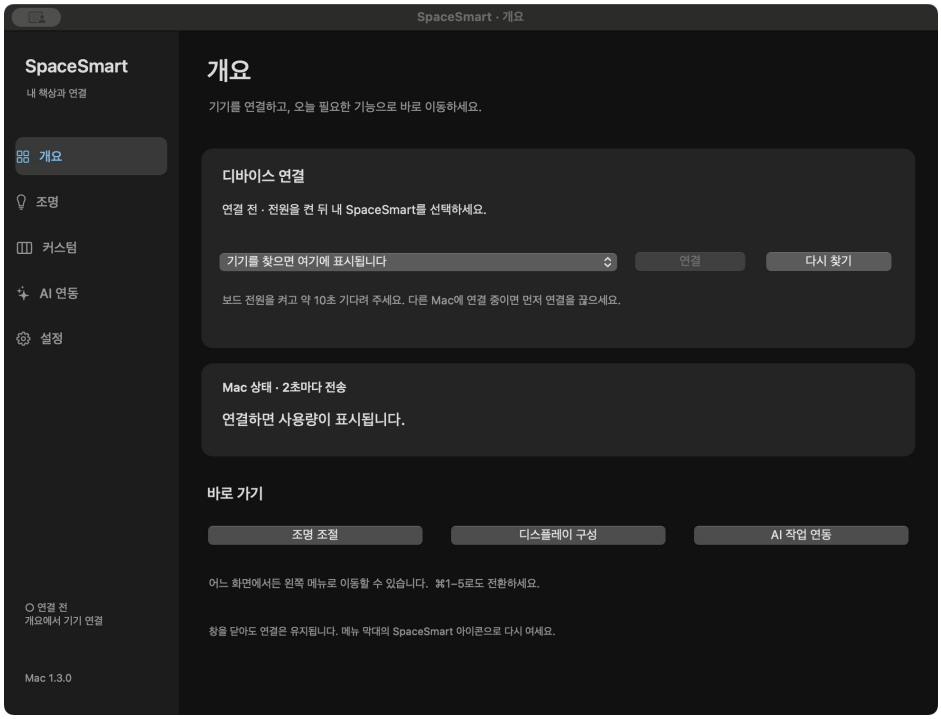
Mac에 앱 복사 → Bluetooth 허용 → 내 디바이스 선택

준비물 macOS 13 Ventura 이상, Bluetooth를 지원하는 Mac, SpaceSmart v2.3 펌웨어가 올라간 ESP32-S3 디바이스. Apple Silicon·Intel 공용 앱입니다.

1. 앱 복사 SpaceSmart-Mac-1.3.0.dmg를 열고 SpaceSmart.app을 Applications(응용 프로그램) 폴더로 드래그합니다. 복사가 끝나면 디스크 이미지를 추출하세요.

2. 앱 실행 응용 프로그램 폴더에서 SpaceSmart를 엽니다. Bluetooth 권한을 요청하면 허용합니다. Python·Homebrew·Xcode·Arduino 설치는 필요 없습니다. AI 앱 계정이나 API 키를 입력할 필요도 없습니다.

3. 기기 연결 보드 전원을 켜고 약 10초 기다립니다. 앱의 **개요** 목록에서 내 SpaceSmart를 선택하고 **연결**을 누릅니다. 연결되면 CPU·메모리·네트워크·시간이 약 2초마다 갱신됩니다.



기기가 여러 대면 원하는 보드 하나만 켜서 먼저 연결하세요. 한 보드는 한 Mac에서 사용하며, 이전 Mac의 연결을 종료해야 새 Mac에서 검색됩니다.

첫 실행이 차단되면

이 배포본은 Developer ID 서명·Apple 공증을 받지 않은 내부 공유용입니다. 보낸 사람과 파일을 확인한 뒤 실행을 한 번 시도하고, **시스템 설정 → 개인정보 보호 및 보안 → 확인 없이 열기**를 사용하세요. 회사 관리 Mac에서 허용되지 않으면 IT 관리자에게 문의하세요.

Apple 공식 안내: Mac에서 안전하게 앱 열기

Claude·Codex 연동하기

선택 기능입니다. 환경 센서·PC 상태·시계는 AI 연동 없이 사용할 수 있습니다.

Claude Desktop: Code 탭

왼쪽 **AI 연동**에서 **Claude 연동 설치**를 누릅니다. Claude Desktop의 **Code 탭**에서 새 작업을 시작하세요. 기존 세션에 설정이 반영되지 않으면 작업을 저장한 뒤 Claude를 다시 엽니다. 일반 Chat·Cowork 상태는 이 방식으로 수집하지 않습니다.

Codex: 설치 후 훅 신뢰 설정

왼쪽 **AI 연동**에서 ① **Codex 연동 설치** → ② **Codex 훅 검토 열기** → ③ 열린 터미널에서 **/hooks** 입력 → ④ **SpaceSmartAgent hook codex** 명령이 들어 있는 새 훅을 검토·신뢰합니다.

> Codex는 새 훅 또는 변경된 훅을 사용자가 승인하기 전까지 실행하지 않습니다. 설치 앱이 이 승인 절차를 대신하거나 건너뛰지 않습니다. 이 과정은 최초 설치 또는 훅 정의가 바뀔 때 필요합니다.

검토 창을 찾지 못하면 Codex를 설치·업데이트하세요. 별도로 Codex CLI를 사용한다면 해당 터미널에서 **codex**를 열고 **/hooks**로 검토할 수 있습니다. 연동 후 새 실제 작업에서 상태가 바뀌는지 확인하세요.

디바이스 표시만 먼저 시험하기

디바이스 노브를 돌려 **AI 작업** 화면을 선택한 뒤 Mac 앱의 **AI 연동 → Codex 표시 시험**을 누릅니다. “상태 전송 시험”이라는 이름으로 작업 중 → 입력 대기 → 응답 완료가 각각 12초 표시됩니다. 총 36초 후 실제 관측 상태로 돌아옵니다.

이 시험은 Bluetooth 전송과 화면 표시를 확인합니다. 실제 Codex 작업 훅이 동작하는지까지 확인하는 시험은 아닙니다. 실제 이벤트가 아직 없으면 시험 뒤 “상태 미확인”이 정상입니다.

표시	의미
작업 중 / 입력 대기	작업 이벤트 수신 / 권한·입력 요청 관측
응답 완료 / 오류 / 중단	종료 이벤트 관측. 60초 후 대기로 전환
상태 미확인	이벤트 없음, PC 연결 중단, 또는 5분간 작업 이벤트 없음

기본 사용자 설정 위치인 ~/.claude/settings.json 및 ~/.codex/hooks.json에 설치합니다. 별도 프로필·사용자 지정 설정 폴더는 이 앱의 자동 설치 대상이 아닙니다. 관리 정책으로 개인 훅이 제한된 환경은 해당 관리자 설정을 따릅니다.

Codex 공식 훅 문서 · Claude Code 공식 훅 문서

매일 쓰는 방법과 문제 해결

Mac은 하나의 창에서 이동합니다. 디바이스는 가로 320×240 · 블랙 테마 4개

Mac 메뉴 개요: 기기 연결·PC 상태 / 조명: 직접 조절·자동 조절·연결 설정 / 커스텀: 카드 3개 구성 / AI 연동: 설치·표시 시험 / 설정: 자동 실행·가이드·데이터. 어느 화면에서도 왼쪽 메뉴 또는 Command+1~5로 바로 이동합니다. 조명 세부 메뉴는 스크롤해도 유지됩니다.

조작	동작
노브 회전 / PUSH 짧게	화면·항목 이동 / 선택·실행
PUSH 0.8초	설정 열기
K0 짧게 / 길게	뒤로 / 홈. 집중 타이머 선택 중 길게 누르면 초기화
그래프 선택 후	노브: 센서 선택. PUSH: 30분·6시간·24시간 전환
테마 변경	설정 → 테마 → PUSH → 회전 → PUSH. 민트·블루·앰버·바이올렛

화면 순서: 실내 환경, 환경 그래프, 컴퓨터, 커스텀, AI 작업, 집중 타이머, 시계, 기기 진단, 설정, 조명. 테마·방향·LED 밝기·집중 시간은 마지막 변경 2초 후 저장합니다. 그래프 이력과 실행 중인 타이머는 재부팅하면 초기화됩니다.

Mac 앱을 계속 사용하려면

창을 닫아도 메뉴 막대의 디스플레이 아이콘에서 계속 실행됩니다. 원하면 **설정 → Mac 로그인 시 자동 실행**을 체크하세요. 앱 종료는 메뉴 막대 아이콘 → **SpaceSmart 종료**입니다. 자동 실행은 기본으로 꺼져 있습니다.

증상	먼저 해볼 일
기기가 안 보임	보드 전원·Mac Bluetooth 확인 → 10초 대기 → 다시 찾기. 다른 Mac·이전 연결 스크립트의 연결을 종료합니다.
Bluetooth 권한을 거절함	시스템 설정 → 개인정보 보호 및 보안 → Bluetooth에서 SpaceSmart 허용 후 앱을 다시 실행합니다.
연결됐는데 시간이 안 나옴	앱의 CPU·메모리 숫자가 갱신되는지 확인합니다. 디바이스가 v2.3인지 확인하고 연결을 다시 시도합니다.
AI가 계속 미확인	해당 연동 설치, Codex 훅 신뢰, 새 실제 작업 여부를 확인합니다. 작업 저장 후 AI 앱을 다시 열어 봅니다.
화면이 안 보임 / 센서 --	보드용 펌웨어·배선·PSRAM 설정을 확인합니다. 초기화 중에는 약 10초, SCD40의 첫 값은 추가 대기가 필요할 수 있습니다.

PC 데이터가 6초, AI 데이터가 15초 끊기면 디바이스는 수신 대기/미확인으로 돌아갑니다. Mac 메모리는 활성·유선·압축 페이지 합계, 네트워크는 en* 인터페이스 합계로 다른 시스템 모니터와 집계 기준이 다를 수 있습니다.

새 보드에 펌웨어 올리기

조명 기능에는 v2.3 이상이 필요합니다. 이전 펌웨어는 업데이트하세요.

배포받은 **SpaceSmart-Source-1.3.0.zip**을 풉니다. Arduino IDE에서 **Arduino/SpaceSmart/SpaceSmart.ino**를 여세요. 같은 폴더의 .h·.cpp 파일을 모두 유지해야 합니다. 펌웨어를 올릴 때만 Arduino 도구가 필요합니다.

보드 매니저 추가 URL: https://espressif.github.io/arduino-esp32/package_esp32_index.json
보드 매니저에서 **esp32 by Espressif Systems 3.3.7**을 설치합니다.

Arduino IDE 항목	값
Board	ESP32S3 Dev Module
Flash Size / Mode	16MB / QIO 80MHz
Partition Scheme	16M Flash (3MB APP/9.9MB FATFS)
PSRAM	OPI PSRAM
USB Mode / USB CDC On Boot	Hardware CDC and JTAG / Enabled
Upload Mode / Speed	UART0 / Hardware CDC / 921600 (실패 시 115200)

라이브러리 매니저에서 설치

Adafruit GFX Library **1.12.6**, Adafruit ST7735 and ST7789 Library **1.11.0**, Adafruit NeoPixel **1.15.5**, Adafruit VEML7700 Library **2.1.6**, Adafruit BusIO **1.17.4**, Sensirion I2C SCD4x **1.1.0**, Sensirion Core **0.7.3**, RF24 by TMRh20 **1.6.2**. 의존성 설치 안내가 나오면 함께 설치합니다. Id2410 라이브러리는 통합 v2.3에서 필요 없습니다.

업로드와 첫 확인

데이터 통신이 되는 USB 케이블로 **ESP32 개발보드의 USB 포트**를 연결하고 해당 포트를 선택해 업로드합니다. 외부 전원 분배용 USB-C 단자로는 업로드할 수 없습니다. 인식되지 않으면 BOOT를 누른 상태에서 RESET을 눌렀다 떼고 BOOT를 놓은 뒤 새 포트를 선택합니다.

업로드 후 화면이 먼저 나오고 LED → 환경 센서 → 레이더 → nRF24 → Bluetooth 순으로 켜집니다. 시리얼 모니터는 **115200 baud / Newline**이며 **status**로 진단할 수 있습니다. LCD는 ST7789 240×320 패널을 가로 320×240으로 사용합니다.

> 이 코드는 검증된 ESP32-S3 N16R8와 기존 SpaceSmart 배선용입니다. 다른 보드는 Config.h와 Arduino/wiring/signal-connections.csv를 먼저 대조하세요. nRF24로 MJGJD01YL 조명 제어를 지원합니다. 조명 미설치로 실제 RF 동작은 검증 전이며, 첫 등록과 실물 확인 절차는 다음 조명 안내를 따르세요.

Arduino IDE · Espressif 보드 패키지 설치 안내

커스텀 탭과 LED 시험

새 기능 · 펌웨어 v2.3와 Mac 앱 1.3.0을 함께 사용하세요.

큰 시계와 하단 환경 값

시계 화면은 가로 기준 96px의 HH:MM 숫자를 표시합니다. 기존 PC 동기화 안내 문구는 제거하고 온도·습도·CO_2를 하단의 세 카드에 큰 숫자로 배치했습니다. 시간 동기화 방식은 이전과 같습니다.

커스텀 항목 3개 선택

왼쪽 **커스텀**에서 카드 1~3의 데이터 종류를 선택하고 **변경 사항 저장**을 누릅니다. 메뉴를 옮겨도 작성 내용은 유지되며 저장해도 화면은 닫히지 않습니다. 연결 전에도 저장할 수 있습니다. 디바이스 노브로 **커스텀** 탭을 열면 선택한 항목이 자동 갱신됩니다. 기본값은 모두 사용 안 함입니다.

형식	선택할 수 있는 데이터
사용률 막대	CPU, 메모리, 저장 공간 사용률. 제목·백분율·0~100% 막대 표시
자동 텍스트	네트워크 RX/TX 또는 저장 공간 여유/전체. 제목 아래 좌·우 값 표시
사용자 텍스트	제목, 하단 왼쪽, 하단 오른쪽을 직접 입력
사용 안 함	해당 칸 비우기

제목은 비워두면 데이터 종류의 기본 이름을 씁니다. 제목·좌·우 텍스트는 각각 UTF-8 24바이트(한글 약 8자)까지 전송되며 화면 폭을 넘으면 말줄임표로 표시합니다. 설정은 Mac에 저장됩니다. 선택하지 않았거나 10초 동안 값이 들어오지 않은 카드는 안내 문구 없이 비워둡니다.

PC 없이 LED 연결 시험

PUSH 길게 → 설정 → LED 시험 → PUSH 순서로 엽니다. 노브로 자동 순환·빨강·초록·파랑·흰색·한 개씩 중 하나를 고르고 PUSH로 시작합니다. 본체 LED 8개와 신호등 LED 1개를 함께 시험합니다.

> 시험은 밝기 20으로 최대 15초 실행한 뒤 원래 표시와 밝기로 복원합니다. PUSH를 다시 누르면 중지, K0는 즉시 중지하고 설정으로 돌아갑니다. Bluetooth·AI 앱 연결이 없어도 동작합니다.

LED의 전기적 연결 상태를 자동 판정하는 기능은 아닙니다. 실제 LED가 켜지는지 눈으로 확인하세요. “한 개씩”은 본체 LED부터 신호등까지 차례로 점등하므로 끊긴 위치를 확인하기 좋습니다.

다른 데이터로 확장하려면

같은 BLE 수신 경로에 custom 명령을 추가했습니다. 그래프형은 제목·0~100 사용률, 텍스트형은 제목·좌·우 텍스트를 받습니다. 전체 형식과 길이 제한은 소스 ZIP의 **Arduino/docs/CUSTOM_PROTOCOL.txt**에 있습니다. 기존 PC·시계·AI 명령도 그대로 동작합니다.

MJGJD01YL 조명 연결

펌웨어 v2.3 · Mac 앱 1.3.0 · 실제 조명 설치 후 순서대로 확인하세요.

현재 조명 제어 코드는 프로토콜·화면·로직·빌드를 검사한 상태입니다. 실제 조명이 아직 설치되지 않아 무선 송수신과 현장 자동 조절은 검증 전입니다. 조명 제어와 자동 조절은 기본으로 꺼져 있습니다.

원래 리모컨과 함께 사용하기

① 센서 위치를 고정하고 조명을 설치합니다. ② Mac 앱의 **조명 → 연결 설정 → 리모컨 ID 등록**을 누릅니다. ③ 60초 안에 원래 리모컨을 두 번 조작합니다. 같은 ID의 정상 패킷 두 개를 받으면 등록됩니다. 주변 다른 리모컨은 이때 조작하지 마세요.

④ **연결 설정**에서 현재 켜짐/꺼짐을 맞춥니다. ⑤ **직접 조절 → 기본 조명 복원**으로 밝기·색온도를 맞춥니다. ⑥ 실제 동작을 확인한 뒤 **연결 설정 → 설치 후 실제 조명 동작을 확인함**을 선택합니다.

제어	사용법
전원	전원 전환은 켜짐/꺼짐 토글입니다. 상태가 다르면 현재 켜짐/꺼짐으로 맞춥니다.
밝기	슬라이더 또는 +/-로 16단계 조절. 0단계는 최소 밝기이며 꺼짐이 아닙니다.
색온도	따뜻/차갑게 또는 슬라이더. 2700~6500K 표시는 단계의 대략적인 환산값입니다.
기본 조명 복원	원래 리모컨 길게 누르기: 약 50% 밝기, 2700K.

디바이스만으로 조절하기

노브로 **조명** 화면 → PUSH → 항목 선택 → PUSH. 밝기·색온도는 노브로 조절하고 PUSH 또는 K0로 편집을 끝냅니다. **조명 설정**에서 리모컨 등록·전원 상태 맞춤·실물 확인을 할 수 있습니다.

새 ID로 페어링해야 할 때

Mac의 **조명 → 연결 설정 → 고급**에서 6자리 ID를 저장하고 조명 제어를 켭니다. 조명 전원을 다시 연결한 뒤 20초 안에 **새로 페어링**을 실행합니다. 디바이스에서는 **조명과 새로 연결**을 두 번 눌러 실행합니다. 다른 ID로 연결하면 기존 리모컨 연결이 바뀝니다. 원래 리모컨과 함께 쓰려면 위의 ID 등록 방식을 사용하세요.

조명은 명령 수신 여부나 현재 상태를 회신하지 않습니다. 표시값은 송신·리모컨 기록에 따른 추정입니다. 재부팅·무선 오류·리모컨 이벤트 누락 후에는 상태를 다시 맞추세요. MJGJD02YL/1S는 지원하지 않습니다.

공개 프로토콜 분석 · ESP32 구현 참고 · MIT 고지는 소스의 Arduino/docs/LIGHTBAR_THIRD_PARTY.txt에 있습니다.

내 환경에 맞춘 밝기 학습

고정 lux 목표 대신, 설치 위치별로 사용자가 고른 밝기를 기록합니다.

조정 전후를 기록합니다

디바이스·Mac·등록한 원래 리모컨의 연속 조작을 한 번으로 묶습니다. 조정 전 최근 5개 조도의 중앙값과, 마지막 조작 뒤 안정화 시간을 둔 5개 조도의 중앙값을 비교합니다. 마지막 조작 12초 후 기록을 확정합니다.

센서 오류·포화·큰 조도 변동·색온도 동시 변경·전원 상태 미확인·실물 미검증 기록은 학습에서 제외합니다. 자동 조절 결과는 별도로 기록하고 사용자 선호를 학습하는 자료에는 넣지 않습니다.

위치 프로필을 선택하세요

Mac의 **조명 → 자동 조절**에서 “창가 책상”, “저녁 작업실”처럼 위치 이름을 선택하거나 새로 만듭니다. 기기·조명 ID·위치별로 기록을 나눕니다. 센서나 조명을 옮기면 새 위치를 만들어 다시 학습하세요.

자동 조절 준비 조건

조건	의미
유효 기록 20개 이상	안정된 조도와 확인된 조명 상태에서 사용자가 밝기를 조절한 기록
사용 시간 3회 이상	20분 이상 간격으로 나뉜 사용 시간. 한 번의 노브 회전은 한 묶음
밝기 단계 3개 이상	다양한 선호 기록과 일관된 단계별 조도 변화
예측 일관성	기록 하나를 제외한 나머지 자료로 사용자 선택을 충분히 재현

준비 완료 후 **학습한 선호로 밝기 자동 조절**을 직접 켭니다. 실제 동작 확인·켜진 조명·재실·신선하고 안정된 조도·학습한 주변 조건이 맞을 때만 최소 90초 간격으로 한 단계씩 조절합니다. 전원과 색온도는 자동으로 바꾸지 않습니다.

수동 조작 뒤 10분간 기다립니다. 예상과 다른 조도 반응이 2회 연속이면 자동 조절을 끕니다. 앱 종료·연결 끊김·기기 재부팅 뒤에는 자동 조절을 다시 직접 켜야 합니다. 처음 보는 주변 밝기에서는 추가 수동 기록을 기다립니다.

기록 보관과 확인

Mac 없이 조절한 기록은 디바이스에 최대 32건 보관하고, 연결 후 Mac 저장이 확인되면 지웁니다. 초과하면 오래된 기록부터 빠집니다. 이전 부팅의 시각을 모르는 기록은 학습에서 제외합니다. Mac은 위치별 최대 2000건을 보관하며 **기록 CSV 저장**으로 내보낼 수 있습니다.

저장 위치: 데이터 폴더의 **light-learning.json**. 자세한 규칙과 전송 형식은 소스 ZIP의 Arduino/docs/LIGHTBAR_KO.txt와 LIGHTBAR_PROTOCOL.txt를 참고하세요. 학습 조건 통과는 실제 환경에서의 성능 보장을 뜻하지 않습니다.

업데이트·삭제·배포 안내

개인 설정 백업과 실시간 상태 파일은 다른 사람에게 보내지 마세요.

업데이트

메뉴 막대에서 SpaceSmart를 종료하고 새 앱으로 교체합니다. AI 상태 수집기도 갱신하려면 사용하는 앱의 **연동 설치**를 다시 누릅니다. 기존 SpaceSmart 훅은 중복 추가하지 않습니다. Codex에서 변경된 훅 검토를 요구하면 /hooks에서 직접 확인합니다.

삭제

① **AI 연동 → AI 연동 해제**를 누릅니다. ② **설정 → Mac 로그인 시 자동 실행**을 해제합니다. ③ 메뉴 막대에서 앱을 종료합니다. ④ 응용 프로그램의 SpaceSmart를 휴지통으로 옮깁니다. 백업과 상태 파일까지 지우려면 앱의 **설정 → 데이터 폴더 열기**를 미리 열어두고 해당 SpaceSmart 폴더만 삭제하세요.

데이터 폴더: **~/Library/Application Support/SpaceSmart**
앱 설정은 macOS 사용자 기본 설정(local.spacesmart.mac)에 남을 수 있습니다. 기존의 별도 개발용 Start-SpaceSmart.command/Python 연동을 쓰던 컴퓨터는 그 연결을 먼저 종료하세요. 새 앱은 이전 개발용 훅을 임의로 지우지 않습니다.

개인정보와 원상 복구

수집 정보는 CPU·메모리·네트워크 사용량, 시각, AI 상태, 프로젝트 폴더 이름, 이벤트 시각·프로세스 식별값입니다. 프롬프트·응답 본문·도구 인수·API 키를 수집하지 않고 클라우드로 전송하지 않습니다. 측정값은 선택한 SpaceSmart로 Bluetooth 전송됩니다. 조명 기능에서는 기기 ID·위치 이름·조도·조명 조정 전후 값과 시각을 이 Mac에 저장합니다.

AI 설치·해제는 SpaceSmart가 추가한 훅만 관리하며 다른 훅·권한·설정은 보존합니다. 변경 전 원본은 데이터 폴더의 **backups**에 저장합니다. 이 원본에는 기존 개인 설정이 들어 있을 수 있으므로 공유하지 마세요. 복구할 때에는 AI 앱을 종료하고, 백업 이후 추가한 다른 설정이 없는지 확인한 다음 해당 파일을 복원하세요.

배포하는 분에게

일반 사용자는 **DMG와 이 PDF**만 받으면 됩니다. 새 보드 업로드나 개발이 필요한 사람에게는 소스 ZIP도 전달하세요. 앱은 macOS 13 이상용 arm64+x86_64 Universal 실행 파일이며 받는 사람이 빌드할 필요가 없습니다. 소스 재빌드에는 Xcode Command Line Tools가 필요합니다.

이 패키지는 ad-hoc 코드 서명을 사용하며 Apple 공증은 되어 있지 않습니다. 경고 없는 외부 배포에는 배포자의 Developer ID 인증서와 Apple 공증이 추가로 필요합니다. Gatekeeper 전체 해제나 보안 설정 우회 명령은 사용하지 않습니다.

검증 범위

설치·재설치·해제 시 기존 설정 보존, 잘못된 JSON 보호, 상태 수집·개인정보 필터·만료, 경로의 공백·한글·따옴표, Universal 빌드와 코드 서명을 검사했습니다. v1.3의 단일 창 메뉴 이동·단축키·커스텀 저장·스크롤을 확인했습니다. 현 Mac에서 Bluetooth 권한 허용 후 실제 기기 연결과 PC 값 갱신을 확인했고, 추가 화면과 커스텀·LED 시험 로직을 검사했습니다. Intel 실물, macOS 13 실물, 새 사용자 계정의 Gatekeeper 흐름과 실제 Claude/Codex 전체 작업 수명주기는 별도 확인 대상입니다.